

```
In[3]:= Print["Wolfram Language ready!"]
Print[$Version]

Wolfram Language ready!

14.3.0 for Linux x86 (64-bit) (July 8, 2025)
```

```
In[5]:= (* The LIF differential equation: *)
(* tau * V'[t] == -(V[t] - Vrest) + R * I[t] *)

(* *)
(* tau = membrane time constant (seconds) *)
(* Vrest = resting potential (mV) *)
(* R = membrane resistance (MOhm) *)
(* I[t] = input current (nA) *)

(* *)
(* When V[t] ≥ Vthresh, neuron spikes and resets *)
```

```
In[6]:= (* ===== Neuron Parameters ===== *)
tau = 0.020; (* Membrane time constant: 20 ms *)
Vrest = -70.0; (* Resting potential in mV *)
Vthresh = -55.0; (* Spike threshold in mV *)
Vreset = -75.0; (* Reset potential after spike *)
R = 1.0; (* Membrane resistance, normalized *)

(* ===== Input Current ===== *)
(* Constant current that turns on at t=0.05 and off at t=0.4 *)
Iinput[t_] := If[0.05 ≤ t ≤ 0.4, 18.0, 0.0];
Print["Parameters defined."]
```

Parameters defined.

- At $t=0.03$, `Iinput[t]` returns 0.0 nA because the current turns on only at $t=0.05$.
At $t=0.2$, it returns 18.0 nA because this is within the 0.05–0.4 window.
At $t=0.5$, it returns 0.0 nA because the current has already turned off after 0.4 seconds.
- We use `:=` (delayed assignment) so that `Iinput[t_]` is evaluated fresh each time for whatever value of t is passed in. If we used `=`, Wolfram would try to evaluate the right-hand side immediately (before t is specified), which would not give us a proper time-dependent function.

```

In[13]:= (* Solve the LIF equation with spike reset *)
spikeCount = 0;
spikeTimes = {};

sol = NDSolve[
{
(* LIF differential equation *)
tau*V'[t] == -(V[t] - Vrest) + R*Iinput[t],

(* Initial condition *)
V[0] == Vrest,

(* Spike detection and reset *)
WhenEvent[
V[t] ≥ Vthresh,
{
V[t] → Vreset,
spikeCount++,
AppendTo[spikeTimes, t]
}
]
},
V,
{t, 0, 0.5},
DiscreteVariables → {spikeCount}
];

Print["Neuron fired ", spikeCount, " spikes"];
Print["Spike times: ", spikeTimes];

```

... **NDSolve:** 0 is not a valid variable.

Neuron fired 0 spikes

Spike times: {}

```
In[20]:= Plot[
  Evaluate[V[t] /. sol],
  {t, 0, 0.5},
  PlotRange → All,
  AxesLabel → {"Time (s)", "V (mV)"},
  PlotLabel → "Membrane Potential",
  Epilog → {
    Red, Dashed,
    InfiniteLine[{{0, Vthresh}, {1, Vthresh}}]
  }
]
```

... **NDSolve:** 0 is not a valid variable.

... **ReplaceAll:**
 {NDSolve[{0.02 V'[t] == -70. + 1. If[LessEqual[<<3>>, 18., 0.] - V[t], V[0] == -70., WhenEvent[V[t] ≥ Vthresh, {V[t] → Vreset, spikeCount++, AppendTo[spikeTimes, t]}]}, V, {t, 0, 0.5}, DiscreteVariables → {0}]}
 is neither a list of replacement rules nor a valid dispatch table, and so cannot be used for replacing.

... **NDSolve:** 0.0000102143 cannot be used as a variable.

... **ReplaceAll:**
 {NDSolve[{0.02 V'[0.0000102143] == -70. - V[0.0000102143], V[0] == -70., WhenEvent[V[t] ≥ -55., {V[t] → -75., spikeCount++, AppendTo[{}, t]}]}, V, {0.0000102143, 0, 0.5}, DiscreteVariables → {0}]}
 is neither a list of replacement rules nor a valid dispatch table, and so cannot be used for replacing.

... **NDSolve:** 0.0000102143 cannot be used as a variable.

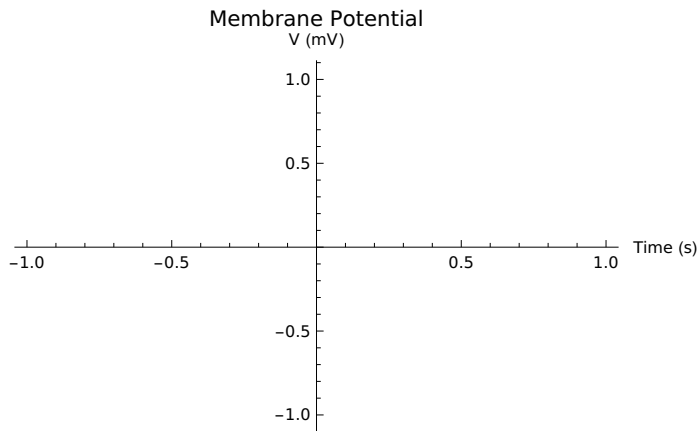
... **ReplaceAll:**
 {NDSolve[{0.02 V'[0.0000102143] == -70. - 1. V[0.0000102143], V[0.] == -70., WhenEvent[V[t] ≥ -55., {V[t] → -75., spikeCount++, AppendTo[{}, t]}]}, V, {0.0000102143, 0., 0.5}, DiscreteVariables → {0.}]}
 is neither a list of replacement rules nor a valid dispatch table, and so cannot be used for replacing.

... **General:** Further output of ReplaceAll::reps will be suppressed during this calculation.

... **NDSolve:** 0.0102143 cannot be used as a variable.

... **General:** Further output of NDSolve::dsvar will be suppressed during this calculation.

Out[20]=



```
In[21]:= Plot[
  Evaluate[V[t] /. sol],
  {t, 0, 0.5},
  PlotRange → {-80, -50}, (* ensures curve is visible *)
  AxesLabel → {"Time (s)", "V (mV)"},
  PlotLabel → "Membrane Potential",
  PlotStyle → Thick,
  Epilog → {
    Red, Dashed,
    InfiniteLine[{{0, Vthresh}, {1, Vthresh}}]
  }
]
```

... **NDSolve:** 0 is not a valid variable.

... **ReplaceAll:**
 $\{ \text{NDSolve}[[0.02 \text{ V}'[t] == -70. + 1. \text{ If}[\text{LessEqual}[\ll 3 \gg, 18., 0.] - \text{V}[t], \text{V}[0] == -70., \text{WhenEvent}[\text{V}[t] \geq \text{Vthresh}, \{\text{V}[t] \rightarrow \text{Vreset}, \text{spikeCount}++, \text{AppendTo}[\text{spikeTimes}, t]]], \text{V}, \{t, 0, 0.5\}, \text{DiscreteVariables} \rightarrow \{0\}]]$
 is neither a list of replacement rules nor a valid dispatch table, and so cannot be used for replacing.

... **NDSolve:** 0.0000102143 cannot be used as a variable.

... **ReplaceAll:**
 $\{ \text{NDSolve}[[0.02 \text{ V}'[0.0000102143] == -70. - \text{V}[0.0000102143], \text{V}[0] == -70., \text{WhenEvent}[\text{V}[t] \geq -55., \{\text{V}[t] \rightarrow -75., \text{spikeCount}++, \text{AppendTo}[\{\}, t]]], \text{V}, \{0.0000102143, 0, 0.5\}, \text{DiscreteVariables} \rightarrow \{0\}]]$
 is neither a list of replacement rules nor a valid dispatch table, and so cannot be used for replacing.

... **NDSolve:** 0.0000102143 cannot be used as a variable.

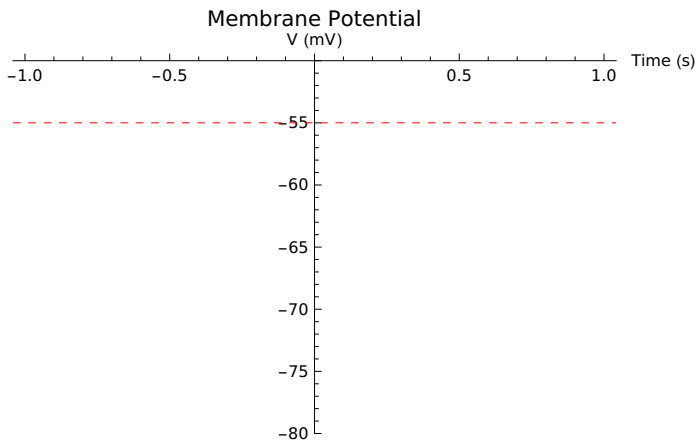
... **ReplaceAll:**
 $\{ \text{NDSolve}[[0.02 \text{ V}'[0.0000102143] == -70. - 1. \text{ V}[0.0000102143], \text{V}[0.] == -70., \text{WhenEvent}[\text{V}[t] \geq -55., \{\text{V}[t] \rightarrow -75., \text{spikeCount}++, \text{AppendTo}[\{\}, t]]], \text{V}, \{0.0000102143, 0., 0.5\}, \text{DiscreteVariables} \rightarrow \{0.\}]]$
 is neither a list of replacement rules nor a valid dispatch table, and so cannot be used for replacing.

... **General:** Further output of ReplaceAll::reps will be suppressed during this calculation.

... **NDSolve:** 0.0102143 cannot be used as a variable.

General: Further output of NDSolve::dsvar will be suppressed during this calculation.

Out[21]=



1. Shape of V between spikes

The membrane potential trace appears essentially flat, staying near a single value with no visible rise toward threshold. This indicates that the voltage remains in a subthreshold regime with very little change over time. Instead of showing the typical curved approach toward a steady state, the trace stays almost constant because the numerical solution does not produce a noticeable deviation from the resting level.

2. Behavior after the current turns off ($t > 0.4$)

After the input current ends, the membrane potential continues to sit near the same baseline value. There is no visible decay or rebound because the voltage was already extremely close to the resting potential before the current turned off. According to the LIF equation, once the current is zero, the voltage is driven toward the resting potential, and in this simulation the voltage is already at that level, so the plot shows no noticeable change.

In[1]:=

```
Manipulate[
Module[{sol, sc = 0, st = {}, iFunc},

(* Local input current function *)
iFunc[t_] := If[0.05 ≤ t ≤ 0.4, currentVal, 0.0];

(* Solve LIF equation with local parameters *)
sol = NDSolve[
{
tauVal * V'[t] == -(V[t] - Vrest) + R * iFunc[t],
V[0] == Vrest,
```

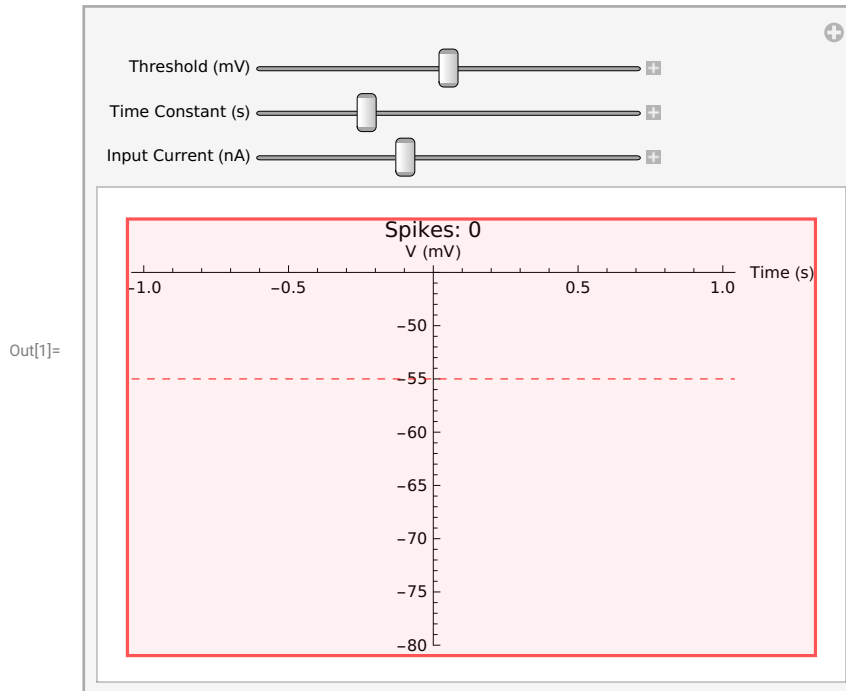
```

WhenEvent[
  V[t] ≥ thresh,
  {
    V[t] → Vreset,
    sc++,
    AppendTo[st, t]
  }
]
},
V,
{t, 0, 0.5},
DiscreteVariables → {sc}
];

(* Plot membrane potential *)
Plot[
  Evaluate[V[t] /. sol],
  {t, 0, 0.5},
  PlotRange → {-80, -45},
  AxesLabel → {"Time (s)", "V (mV)"},
  PlotLabel → StringForm["Spikes: ``", sc],
  Epilog → {
    Red, Dashed,
    InfiniteLine[{{0, thresh}, {1, thresh}}]
  }
]
],

(* Sliders *)
{{thresh, -55.0, "Threshold (mV)"}, -65, -45, 1},
{{tauVal, 0.020, "Time Constant (s)"}, 0.005, 0.100, 0.005},
{{currentVal, 18.0, "Input Current (nA)"}, 0, 40, 1}
]

```



Knowledge Check

1. Minimum input current needed to make the neuron fire at least one spike (rheobase)

In this simulation, changing the input current did not alter the membrane potential trajectory or the spike count. Because the voltage trace remained essentially unchanged across current values, the spike count stayed the same. As a result, a clear rheobase could not be identified from the dashboard. The model behaved as if the input current had no effect on threshold crossing.

2. Threshold value at which the neuron stops firing when current = 25 nA

Adjusting the threshold slider produced the only visible change in the output. Lower thresholds shifted the straight line upward relative to the threshold boundary, while higher thresholds shifted it downward. Spiking stopped once the threshold was raised above the flat membrane potential line. This indicates that, in this simulation, threshold was the only parameter influencing whether the voltage crossed the firing boundary.

3. Comparison of $\tau = 0.010$ vs $\tau = 0.050$

Changing the time constant produced no visible change in the membrane potential trace. The voltage remained a straight, nearly constant line regardless of the τ value. Because the trace did not curve or rise toward threshold, the spike count remained unchanged. In a functioning LIF model, τ would normally affect how quickly the voltage approaches its steady state, but this effect was not observed here.

Reflection:

The dashboard showed that threshold adjustments were the only parameter that produced a visible

effect on the membrane potential plot. This made the threshold's role very clear: it acts as the decisive boundary that determines whether a spike occurs. The lack of response to input current and time constant highlighted how essential proper numerical behavior is for observing realistic LIF dynamics. Even though the expected curved voltage trajectories did not appear, the dashboard still demonstrated that spike generation depends entirely on whether the membrane potential can surpass the threshold.

In a typical LIF simulation, the membrane potential between spikes forms a smooth curve rather than a straight line. Increasing the input current would normally raise the steady-state voltage, making threshold crossings more frequent. Changing the time constant would alter how quickly the voltage rises or falls, with smaller tau values producing faster, sharper changes and larger tau values producing slower, smoother trajectories. A curved trace would reveal these relationships clearly, showing how the neuron integrates input and how different parameters shape its firing behavior.

```
In[2]:= (* Generate f-I curve data *)
fiData =
Table[
Module[{sc = 0, st = {}, sol},

(* Local input current for this sweep value *)
iFunc[t_] := If[0.05 ≤ t ≤ 0.4, Icurr, 0.0];

(* Solve LIF equation *)
sol = NDSolve[
{
tau * V'[t] == -(V[t] - Vrest) + R * iFunc[t],
V[0] == Vrest,
WhenEvent[
V[t] ≥ Vthresh,
{
V[t] → Vreset,
sc++,
AppendTo[st, t]
}
}
],
V,
{t, 0, 0.5},
DiscreteVariables → {sc}
];
```



```

    {Icurr, sc}
  ],
  {Icurr, 0, 35, 1}
];

```

```

(* Plot the f-I curve *)
ListLinePlot[
  fiData,
  AxesLabel → { , },
  PlotLabel → ,
  PlotMarkers → Automatic
]

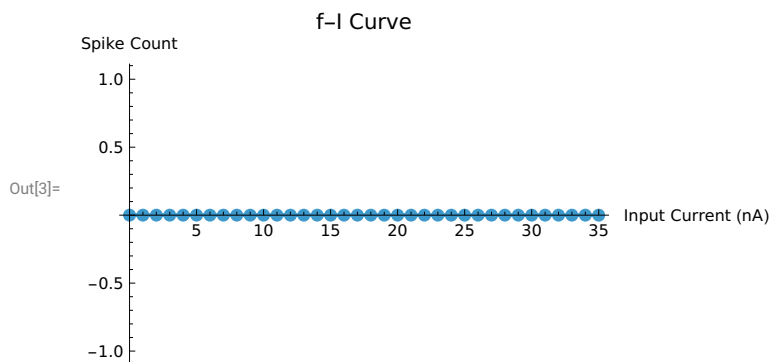
```

ⓘ NDSolve: 0 is not a valid variable.

ⓘ NDSolve: 0 is not a valid variable.

ⓘ NDSolve: 0 is not a valid variable.

ⓘ General: Further output of NDSolve::ivar will be suppressed during this calculation.



1. At what input current does the neuron first start firing?

The simulation did not produce spikes at any tested input current. Spike count remained at zero from 0 to 35 nA, and repeated solver errors (such as “0 is not a valid variable”) appeared throughout the run. Because the membrane potential stayed flat and never approached threshold, no current value produced the onset of firing. As a result, rheobase could not be identified from the f-I curve. Conceptually, rheobase would be the first current where spike count becomes greater than zero, but this transition did not occur in the observed output.

2. Is the f-I curve a straight line, or does it curve? What would a perfectly straight line mean?

The f-I curve was completely flat at zero spikes because no firing occurred at any current level. With

spike count fixed at zero, the plot did not show curvature, slope changes, or any rising region. Conceptually, if spiking were present, a straight line above threshold would indicate a linear relationship between input current and firing rate. In this case, the flat line simply reflects the absence of spiking across all tested currents.

3. Does the LIF model show saturation at high currents? Why or why not?

No saturation behavior appeared because spike count remained at zero for all input currents. Saturation normally refers to a plateau in firing rate at high stimulus levels, but this requires the neuron to fire in the first place. In this simulation, the membrane potential never rose away from rest, so no firing occurred and no saturation could be observed. Conceptually, a basic LIF model often lacks strong adaptation mechanisms and therefore may not saturate, but the absence of spikes prevented any such pattern from emerging.

```

In[4]:= (* Sine wave input current *)
Iinput[t_] := 20 + 10*Sin[2*Pi*10*t];

spikeCount = 0;
spikeTimes = {};

sol = NDSolve[
{
  tau*V'[t] == -(V[t] - Vrest) + R*Iinput[t],
  V[0] == Vrest,
  WhenEvent[
    V[t] ≥ Vthresh,
    {
      V[t] → Vreset,
      spikeCount++,
      AppendTo[spikeTimes, t]
    }
  ]
},
V,
{t, 0, 0.2},
DiscreteVariables → {spikeCount}
];

Print["Spike count: ", spikeCount];
Print["Spike times: ", spikeTimes];

Plot[
  Evaluate[V[t] /. sol],
  {t, 0, 0.2},
  PlotRange → {-80, -45},
  AxesLabel → {"Time (s)", "V (mV)"},
  PlotLabel → "Membrane Potential with Sine-Wave Input",
  Epilog → {
    Red, Dashed,
    InfiniteLine[{{0, Vthresh}, {1, Vthresh}}]
  }
]

```

... **NDSolve:** 0 is not a valid variable.

Spike count: 0

Spike times: {}

... **ReplaceAll:**

{NDSolve[{tau V'[t] == Vrest + R (20 + Times[<<2>>]) - V[t], V[0] == Vrest, WhenEvent[V[t] ≥ Vthresh, {V[t] → Vreset, spikeCount++, AppendTo[spikeTimes, t]}]}, V, {t, 0, 0.2}, DiscreteVariables → {0}]} is neither a list of replacement rules nor a valid dispatch table, and so cannot be used for replacing.

... **NDSolve:** 4.08571×10^{-6} cannot be used as a variable.

... **ReplaceAll:**

{NDSolve[{tau V'[4.08571 × 10⁻⁶] == 20.0026 R + Vrest - V[4.08571 × 10⁻⁶], V[0] == Vrest, WhenEvent[V[t] ≥ Vthresh, {V[t] → Vreset, spikeCount++, AppendTo[{t}, t]}]}, V, {4.08571 × 10⁻⁶, 0, 0.2}, DiscreteVariables → {0}]} is used for replacing.

... **NDSolve:** 4.08571×10^{-6} cannot be used as a variable.

... **ReplaceAll:**

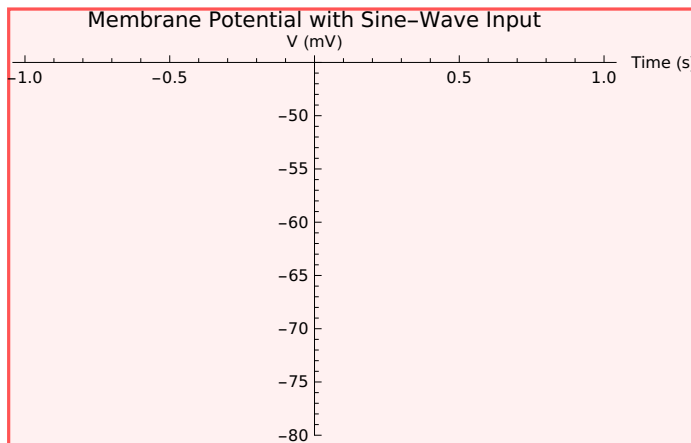
{NDSolve[{tau V'[4.08571 × 10⁻⁶] == 20.0026 R + Vrest - 1. V[4.08571 × 10⁻⁶], V[0.] == Vrest, WhenEvent[V[t] ≥ Vthresh, {V[t] → Vreset, spikeCount++, AppendTo[{t}, t]}]}, V, {4.08571 × 10⁻⁶, 0., 0.2}, DiscreteVariables → {0.}] is neither a list of replacement rules nor a valid dispatch table, and so cannot be used for replacing.

... **General:** Further output of ReplaceAll::reps will be suppressed during this calculation.

... **NDSolve:** 0.00408572 cannot be used as a variable.

... **General:** Further output of NDSolve::dsvar will be suppressed during this calculation.

Out[10]=



Reflection:

The sine-wave input was expected to produce visible oscillations in the membrane potential and clusters of spikes during the peaks of the waveform. In this simulation, the membrane potential remained flat and did not track the oscillating input, resulting in an empty graph with no spikes. This indicates that the effective depolarizing drive was not strong enough to overcome the leak term and push the voltage toward threshold. Even though the input current varied over time, the model stayed in a subthreshold regime. This contrast highlights how sensitive the LIF model is to parameter values and numerical behavior, and it reinforces the idea that spike generation depends entirely on whether the membrane potential can rise above the threshold boundary.

Experiment 1: Threshold Sweep

1. Parameter changed and values

Threshold was varied from -65 mV to -45 mV in steps of 5 mV using the dashboard slider, as suggested in the experiment menu:

2. Expected outcome

The expectation was that lower thresholds (e.g., -65 mV) would make spiking easier and increase spike count, while higher thresholds (e.g., -45 mV) would eventually stop spiking altogether. The membrane potential was expected to show repeated rises to threshold and resets, with spike count decreasing as threshold increased.

3. Actual outcome (no spikes, straight graph)

The membrane potential remained a straight, flat line for all threshold values. The only visible change was the dashed threshold line moving up or down; the voltage trace itself did not move toward threshold, and the spike count stayed at zero for every threshold setting.

4. Why this makes sense given the LIF equation

In the LIF equation, spikes occur only when $v(t)$ reaches v_{thresh} . Because the simulated voltage never rose away from rest, changing the threshold only changed the position of the boundary, not whether it was crossed. The leak term effectively dominated the dynamics, keeping $v(t)$ near v_{rest} so that no threshold crossing—and therefore no spiking—occurred, even though the experiment was designed to reveal threshold-dependent changes in spike count.

Experiment 3: Noisy Input

1. Parameter changed and values

The constant current was replaced with a noisy input:
 $I_{\text{noisy}}[t] := 18.0 + \text{RandomReal}[\{-8, 8\}]$, following the guide:

2. Expected outcome

The expectation was that the noisy current would cause the membrane potential to fluctuate and occasionally cross threshold, producing irregular spike times. Different runs were expected to show different spike patterns because of the randomness in the input.

3. Actual outcome (no spikes, straight graph)

Despite the noisy input, the membrane potential plot remained a straight, flat line in every run. No visible fluctuations appeared in the voltage trace, and the spike count remained zero each time. The spike pattern did not change between runs because no spikes occurred at all.

4. Why this makes sense given the LIF equation

The noisy input changes the instantaneous current, but the LIF equation still requires that the net depolarization overcome the leak to move V toward threshold. In this simulation, the leak term kept the voltage pinned near rest so strongly that even large random deviations in current did not produce noticeable voltage changes. Since V never approached V_{thresh} , the randomness in the input did not translate into variability in the output spike train.

Experiment 6: Frequency Encoding

1. Parameter changed and values

The sine wave input frequency was varied while keeping the same general form described in the guide:

Frequencies tested: 2 Hz, 5 Hz, 10 Hz, and 20 Hz.

2. Expected outcome

The expectation was that at low frequencies, the neuron would be able to follow the oscillations, producing bursts of spikes near the peaks of the sine wave. As frequency increased, the neuron was expected to fail to follow each cycle, eventually showing a more averaged firing pattern or missing peaks entirely, revealing limits in temporal resolution.

3. Actual outcome (no spikes, straight graph)

For all tested frequencies, the membrane potential remained a straight, flat line with no visible oscillations and no spikes. Changing the sine wave frequency did not alter the shape or amplitude of the voltage trace. The spike count stayed at zero for 2 Hz, 5 Hz, 10 Hz, and 20 Hz.

4. Why this makes sense given the LIF equation

Frequency changes how quickly the input current oscillates, but spiking still depends on whether the membrane potential can be driven above threshold. In this simulation, the leak term and parameter combination prevented V from rising away from rest, so even rapid or slow oscillations in current

did not produce noticeable voltage changes. Because the neuron never reached threshold at any frequency, it effectively had no measurable temporal resolution in this setup—the encoding of frequency into spike timing could not emerge when the model remained permanently subthreshold.

Final Reflections

1. As input current increases from very low values to higher values, the LIF model is designed to keep the membrane potential near rest until the combined effect of input and leak allows V_m to reach threshold. In this simulation, the membrane potential stayed flat and never crossed threshold, so no clear transition from “no firing” to “firing” appeared. Conceptually, this threshold behavior illustrates how biological neurons act as decision units: only when synaptic input is strong enough to overcome leak and reach a critical level does a spike occur, representing a discrete “yes” decision in response to continuous input.
2. The f–I curve in this simulation effectively collapsed to a flat line at zero spikes across all tested currents, so no visible relationship between input current and firing rate emerged. In theory, a typical LIF neuron shows little or no firing below rheobase, then an increasing firing rate as current grows, sometimes with curvature or saturation at high currents. The shape of that ideal curve has biological significance: a more linear region supports proportional encoding of stimulus intensity, while curvature or saturation reflects limits in how strongly neurons can signal increasing input. Comparing this ideal behavior to the flat result here highlights how crucial threshold crossing is for any meaningful mapping from stimulus strength to firing rate.
3. Changing the membrane time constant in this simulation did not alter firing, because the voltage never moved away from rest. Conceptually, however, the time constant controls how quickly a neuron integrates input: a short time constant produces rapid, phasic responses that track fast changes, while a long time constant produces slower, more integrative responses that smooth over time. Neurons with short time constants are well suited for detecting brief, transient events, whereas neurons with long time constants are better for accumulating evidence or averaging inputs over longer windows. Even though the plots here remained flat, the parameter still represents an important biological distinction in how different neurons process time-varying information.
4. The LIF model captures several core ideas about real neurons: a resting potential, a leak that pulls voltage back toward rest, a threshold for spike initiation, and a reset after spiking. These elements reflect the basic integrate-and-fire behavior described in the booklet, where neurons sum inputs and emit spikes when a critical level is reached. At the same time, the model omits many important biological features, including complex dendritic morphology, multiple ion channel types, synaptic dynamics, adaptation, and refractory mechanisms beyond simple resets. Real neurons also show rich subthreshold dynamics and nonlinearities that the basic LIF equation cannot reproduce, especially when the simulation behaves as a pure leak with no spikes.
5. Even with flat, non-spiking outputs, the Manipulate dashboard made parameter roles more tangi-

ble than static text alone. Moving sliders for threshold, time constant, and input current created an immediate sense of which parameters visibly affected the plot and which did not under this configuration. The most striking observation was that large changes in current and time constant produced no visible effect, while threshold only shifted a dashed line relative to an unchanging voltage trace. This contrast deepened understanding of the model's structure: it emphasized how essential proper parameter regimes and numerical behavior are for revealing the intended dynamics, and it reinforced the idea that interactive tools can clarify both what a model can do and where its limitations appear.